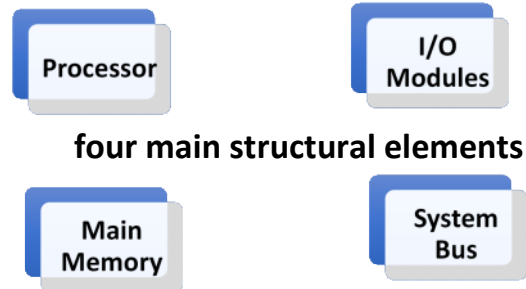


What Does OPERATING SYSTEMS DO ?

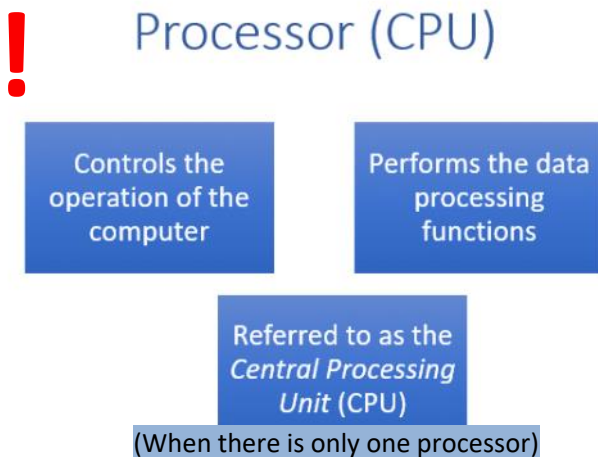
- Use the hardware resources of one or more processors
- Provides a set of services to system users
- Manages secondary memory and I/O devices

Computer Basic Elements

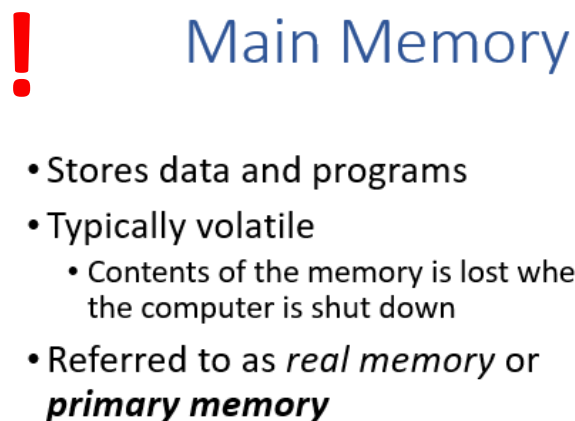


These components are interconnected in some fashion to achieve the main function of the computer, which is to execute programs.

1 - PROCESSOR(CPU)

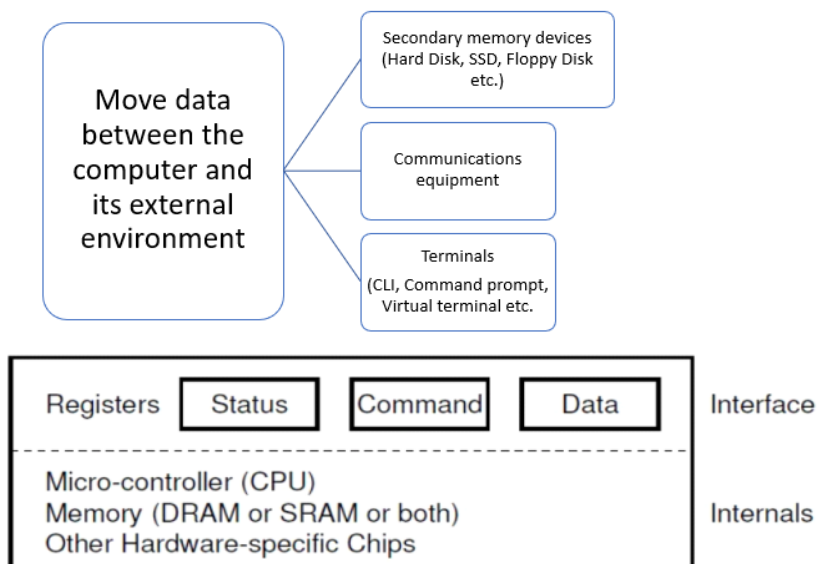


2 - MAIN MEMORY



3 - I/O MODULES

I/O Modules



4- SYSTEM BUS

System Bus

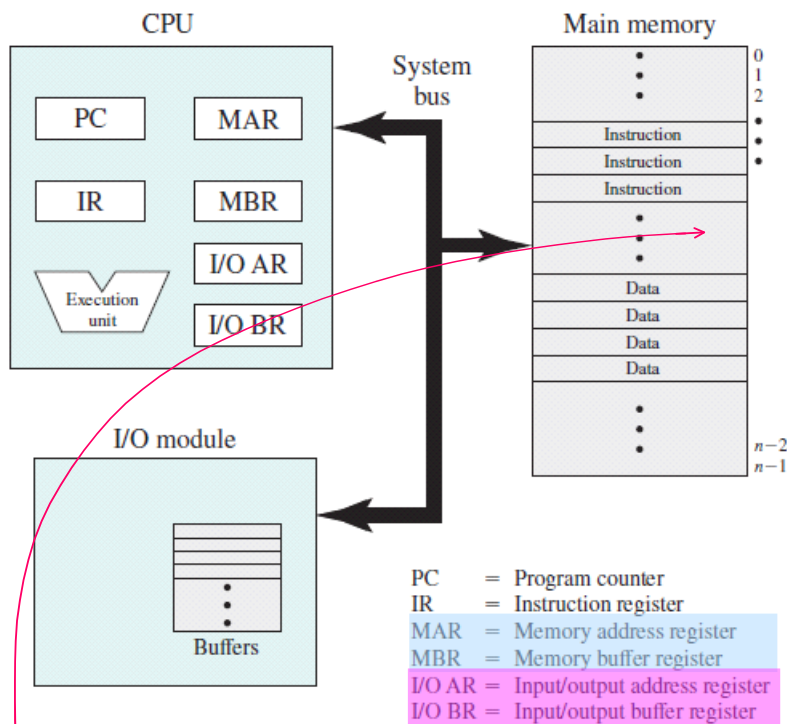
- Provides for communication among processors, main memory, and I/O modules

Status Register : Holds information about the current state of the device (e.g., ready, busy, error)

Command Register : Contains instructions sent by the CPU. (HOLD THE ORDER!)

Data Register: Stores the actual data being transferred between the CPU and the device.

Computer Components: Top-Level View



One of the processor's functions is to exchange data with memory.

For that ; it uses two internal(to the processor) registers :

MAR (Memory Address Register) defines the address in memory for the next read or write operation.

MBR (Memory Buffer Register) : contains the data to be written into memory, or receives the data read from memory.

So, the CPU can access specific memory addresses using the MAR, and transfer data to or from those addresses using the MBR. This provides the CPU with the necessary mechanisms to interact with the memory module as required for its operations.

- **I/OAR (I/O Address Register)**: Specifies a particular I/O (Input/Output) device.
- **I/OBR (I/O Buffer Register)**: Used for the exchange of data between the I/O module and the CPU.

A **memory module** consists of a set of locations, defined by sequentially numbered addresses. Each location contains a bit pattern that can be interpreted as either an instruction or data. An **I/O module** transfers data from external devices to processor and memory, and vice versa. It contains **internal buffers** for temporarily storing data until they can be sent on.

MICROPROCESSOR

Microprocessor

- Invention that brought about desktop and handheld computing
- Contains a processor on a single chip
- Fastest general purpose processors
- Multiprocessors
- **Each chip (socket) contains multiple processors (cores), each with multiple levels of large memory caches, and multiple logical processors sharing the execution units of each core**

DEF : It is a programmable device that takes in input performs some arithmetic and logical operations over it and produces the desired output. In simple words, a Microprocessor is a digital device on a chip that can fetch instructions from memory, decode and execute them, and give results.

GPU

Graphical Processing Units (GPU's)

- **(GPUs)** provide efficient computation on arrays of data using Single-Instruction Multiple Data (**SIMD**) techniques pioneered in supercomputers
- GPU's are no longer used just for rendering advanced graphics
 - Also used for **general numerical processing**
 - Physics simulations for games
 - Computations on large spreadsheets

DSP

Digital Signal Processors (DSPs)

- DSP's are for deal with streaming signals such as audio or video
- DSP's used to be embedded in I/O devices like modems
 - Are now becoming first-class computational devices, especially in handhelds
- Encoding/decoding speech and video (codecs) (co-exist w/ CPU)
- Provide support for encryption and security (co-exist w/ CPU)

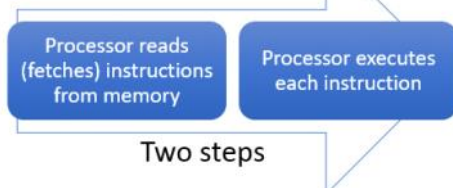
fixed function unit(DSP) & CPU works together.

- Handheld devices are moving away from classic microprocessors and adopting System on Chip (SoC) designs to meet their needs.
- Other components of the system, such as DSPs, GPUs, I/O devices (such as codecs, USB and radios) and main memory, in addition to the CPUs and caches, are on the same chip

Instruction Execution

Instruction Execution

- A program to be executed by a processor consist of a set of instructions stored in memory



Fetch & Execute Stage

Basic Instruction Cycle

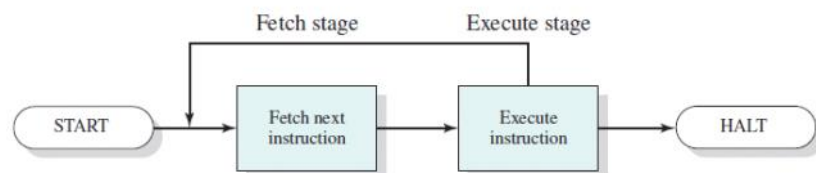


Figure 1.2 Basic Instruction Cycle

- Program execution halts only if the processor is turned off, some sort of unrecoverable error occurs, or a program instruction that halts the processor is encountered.
 - 1-the processor fetches an instruction from memory.
 - 2-the program counter (PC) holds the address of the next instruction to be fetched.
- ⇒ Unless told not to, the processor increases the PC by one after each fetching, so it knows to get the next instruction in order. (to the next higher memory address)

Instruction Register (IR)

Fetch instruction is loaded into Instruction Register (IR). The instruction contains bits that specify the action the processor is to take.



- Processor interprets the instruction and performs required action:
 - Processor-memory
 - Processor-I/O
 - Data processing
 - Control

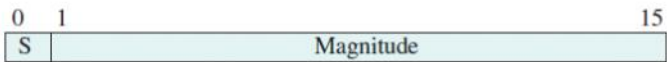
Processor-memory: Data may be transferred from processor to memory or from memory to processor.

- **Processor-I/O:** Data may be transferred to or from a peripheral device by transferring between the processor and an I/O module.
- **Data processing:** The processor may perform some arithmetic or logic operation on data.
- **Control:** An instruction may specify that the sequence of execution be altered. For example, the processor may fetch an instruction from location 149, which specifies that the next instruction be from location 182. The processor sets the program counter to 182. Thus, on the next fetch stage, the instruction will be fetched from location 182 rather than 150.

An instruction's execution may involve a combination of these actions.



(a) Instruction format



(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage
(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory
(d) Partial list of opcodes

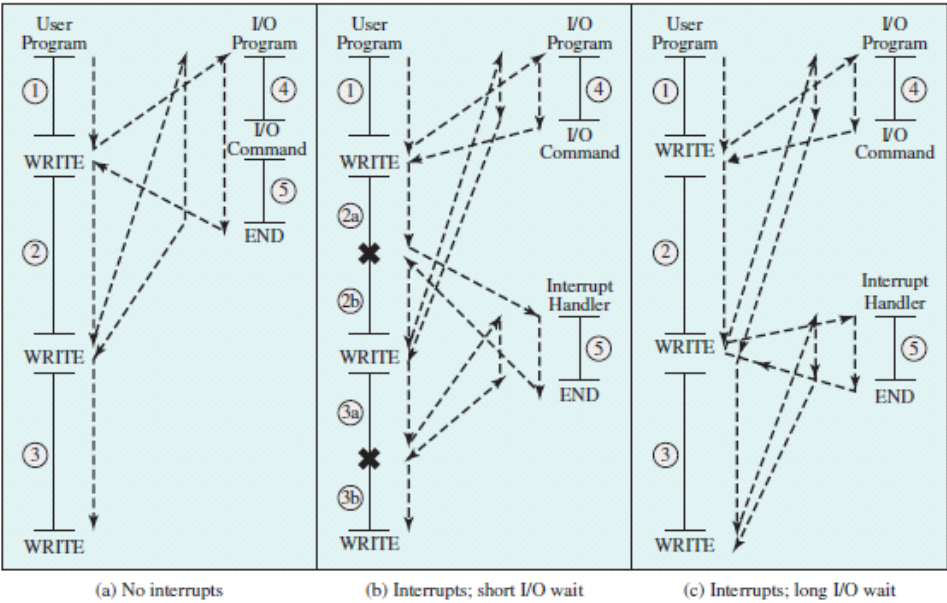
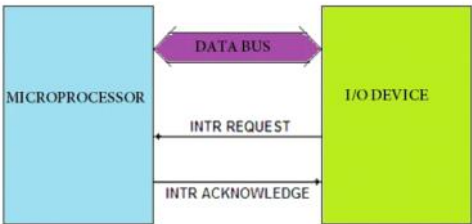
Figure 1.3 Characteristics of a Hypothetical Machine

Consider a simple example using a hypothetical processor that includes the characteristics listed in Figure 1.3. The processor contains a single data register, called the **accumulator (AC)**. Both instructions and data are 16 bits long, and memory is organized as a sequence of 16-bit words. The instruction format provides 4 bits for the opcode, allowing as many as $2^4 = 16$ different opcodes.

The opcode defines the operation the processor is to perform. With the remaining 12 bits of the instruction format, up to 2^{12} words of memory can be directly addressed.

Interrupts

- Mechanism by which other modules may interrupt the normal sequencing of the processor
- Provided to improve processor utilization(usage)
 - Most I/O devices are slower than the processor
 - Processor must pause to wait for device
 - Wasteful use of the processor



✕ = interrupt occurs during course of execution of user program

Figure 1.5 Program Flow of Control Without and With Interrupts

- The **dashed line** represents the processor's execution path, showing the sequence of executed instructions.
- After encountering the first WRITE instruction, the user program is **interrupted**. (temporarily)
- The processor then **switches to the I/O program** execution. (processor completing the input/output task in I/O program before returning to the main program.)
- Once the **I/O program** finishes, the execution **resumes** in the user program, continuing from the instruction

solid vertical lines represent (1, 2, and 3 refer to sequences of instructions that **do not involve I/O**.) segments of code in a program.

Label 4 is a sequence of instruction to prepare for the actual I/O operation.

The actual I/O command. Without the use of interrupts, once this command is issued, the program must wait for the I/O device to perform the requested function

Label 5 is to complete the operation.

immediately after the WRITE instruction.



Program

Generated by some condition that occurs as a result of an instruction execution, such as **arithmetic overflow, division by zero, attempt to execute an illegal machine instruction**, and reference outside a user's allowed memory space.

Timer

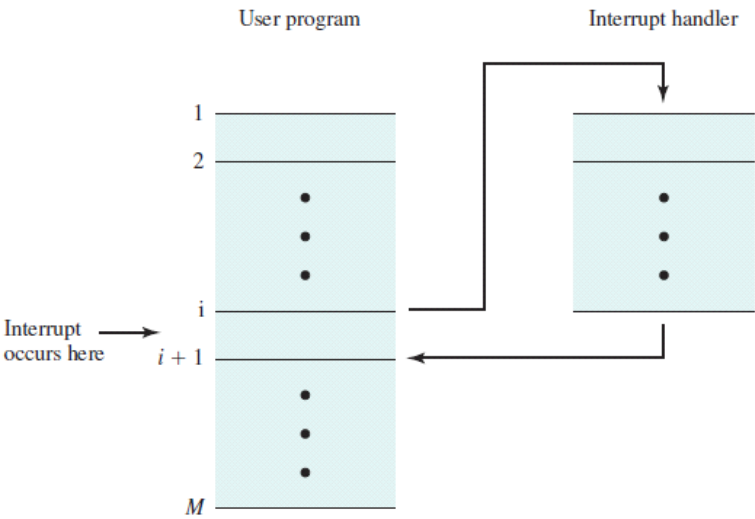
Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.

I/O

Generated by an I/O controller, to signal normal completion of an operation or to signal a variety of error conditions.

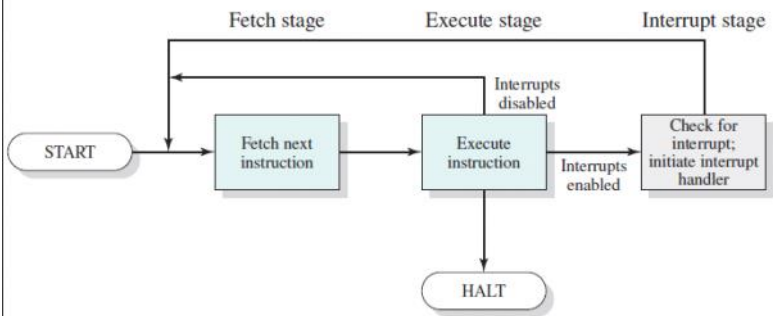
Hardware failure

Generated by a failure, such as **power failure or memory parity error**.



An interrupt temporarily pauses the normal execution of the user program, and once the interrupt processing is completed, the program resumes from where it stopped. The **user program itself doesn't require any special code to handle interrupts**, as the **processor** and **OS** manage suspending and resuming the program at the correct point automatically.

Figure 1.6 Transfer of Control via Interrupts

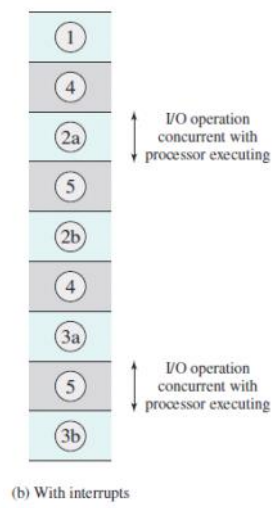
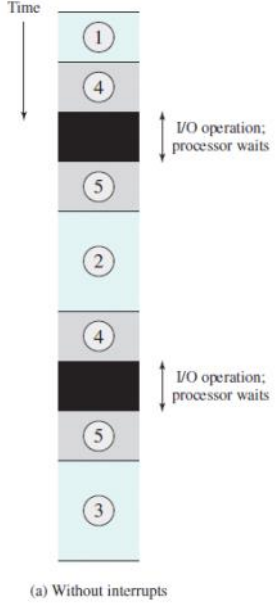


In the interrupt stage, the **processor checks** to see if any interrupts have occurred. If an interrupt is pending, the processor suspends execution of the current program and executes an interrupt handler routine.

Because of the relatively large amount of time that would be wasted by simply waiting on an I/O operation, the processor can be employed much more efficiently with the use of interrupts.

Figure 1.7 Instruction Cycle with Interrupts

Program Timing: Short I/O Wait



(Short I/O Wait):

- **Without interrupts (a):** The processor waits while the I/O operation completes, leading to inefficient use of time.
- **With interrupts (b):** The processor executes user instructions concurrently with the I/O operation, improving efficiency as both tasks happen simultaneously.

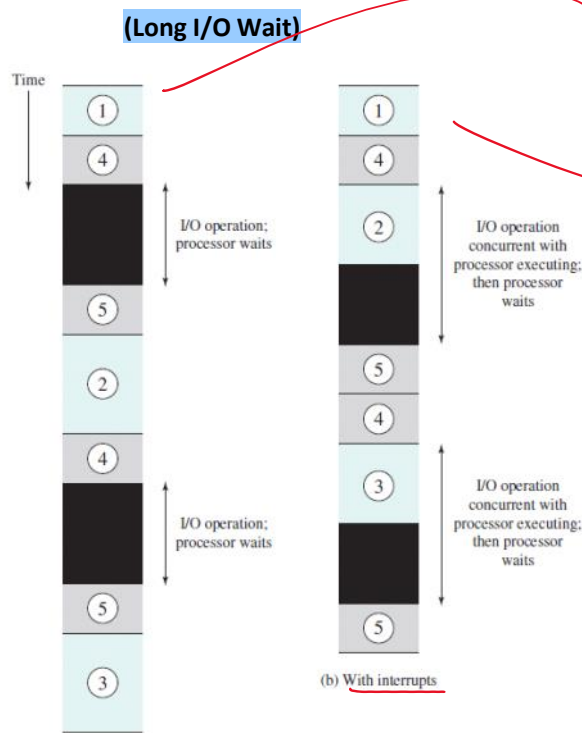


Figure 1.9 Program Timing: Long I/O Wait

- **Without interrupts (a):** For slow devices (e.g., a printer), the processor is stuck waiting for I/O, and the program halts when reaching another WRITE instruction while the previous I/O is still in progress.
- **With interrupts (b):** The processor can perform other tasks while waiting for the I/O operation, making better use of time and improving system efficiency.

Multiple Interrupts

An interrupt occurs while another interrupt is being processed

- e.g. receiving data from a communications line and printing results at the same time

Two approaches:

- Disable interrupts while an interrupt is being processed
- Use a priority scheme

while a printer is generating an interrupt after completing a print operation, a communication line controller may generate another interrupt when new data arrives. This means one interrupt can happen while another is still being processed, allowing different devices to generate interrupts at the same time, such as receiving data and printing results concurrently.

Dealing with Multiple Interrupts

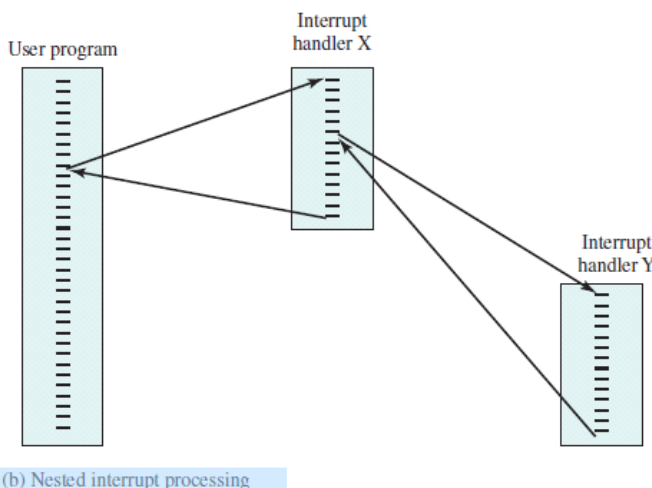
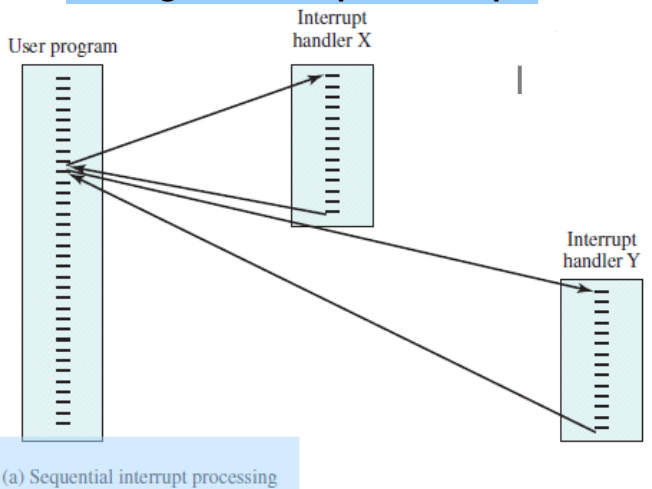


Figure 1.12 Transfer of Control with Multiple Interrupts

1- Disabled Interrupt

Sequential Interrupt Processing:

- When an interrupt occurs, the system disables other interrupts
- Interrupts are processed one at a time in strict order
- Any new interrupts during processing remain pending
- Interrupts are re-enabled after each interrupt handler completes
- Simple but potentially less efficient approach

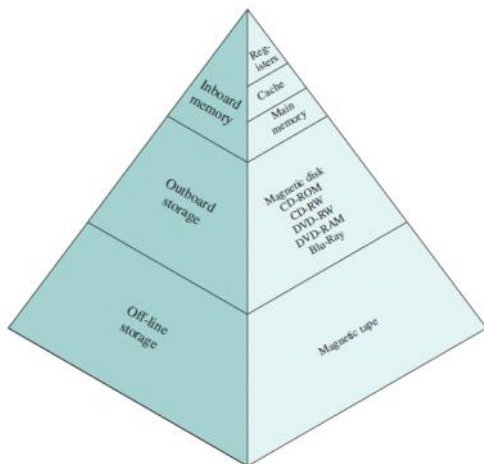
2- Priority Schema

Memory Hierarchy

- Design constraints on a computer's memory
 - How much?
 - How fast?
 - How expensive?
- If the capacity is there, applications will likely be developed to use it.
- Memory must be able to keep up with the processor.
- Cost of memory must be reasonable in relationship to the other components.

There is a trade-off among the three key characteristics of memory: capacity, access time, and cost.

- **Faster access time, greater cost per bit**
- **Greater capacity, smaller cost per bit**
- **Greater capacity, slower access speed**



Going down the hierarchy, following occur:

- Decreasing cost per bit
- Increasing capacity
- Increasing access time
- Decreasing frequency of access to the memory by the processor.

Figure 1.14 The Memory Hierarchy

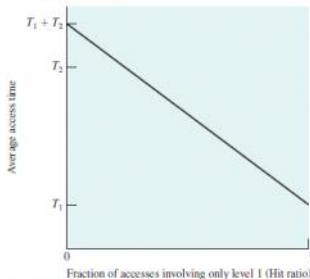


Figure 1.15 Performance of a Simple Two-Level Memory

Principle of Locality

- Memory references by the processor tend to cluster
- Data is organized so that the percentage of accesses to each successively lower level is substantially less than that of the level above
- Can be applied across more than two levels of memory

Cache Memory

- **Invisible** to the OS
- Interacts with other memory management hardware
- Processor must access memory at least once per instruction cycle
- Processor execution is limited by memory cycle time
- Exploit the principle of locality with a small, fast memory

Secondary Devices (auxiliary memory)

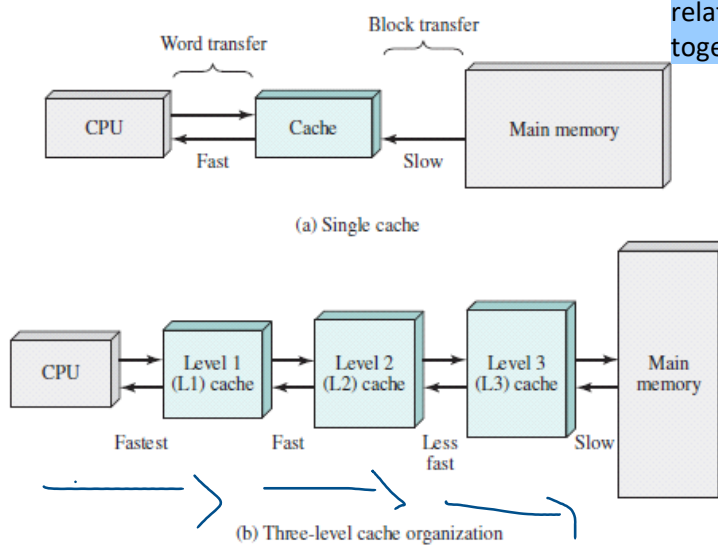
- **External**
- **Nonvolatile**
- **Used to store program and data files**

The processor accesses memory multiple times during instruction cycles, which limits **instruction execution speed due to memory cycle time**. This has become a significant issue because processor speeds have consistently increased faster than memory access speeds. There is a trade-off between speed, cost, and size, and ideally, main memory would use the same technology as processor registers, but this is too costly. **The solution is to use a small, fast cache memory** between the processor and main memory to take advantage of the principle of locality.

Cache Principles

Cache memory aim to provide memory access time approaching that of the fastest memories available, and at the same time support a large memory size that has the price of less expensive types of semiconductor memories.

The cache contains a copy of a portion of main memory. When the processor attempts to read a byte or word of memory, a check is made to determine if the byte or word is in the cache. If so, the byte or word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of bytes, is read into the cache then the byte or word is delivered to the processor. Because of the locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that many of the near-future memory references will be to other bytes in the block.



relatively large and slow main memory together with a smaller, faster cache memory.

Cache Design

- Cache size
- Block size
- Mapping function
- Replacement algorithm
- Write policy
- Number of cache levels

Cache Size : Small caches can have a significant impact on performance.

Block Size : The unit of data exchanged between cache and main memory. When block size increases from very small to larger sizes, the hit ratio initially rises due to the principle of locality. This means that when one piece of data is accessed, nearby data is also likely to be accessed soon. Larger block sizes bring more useful data into the cache.

However, if the block size gets too big, the hit ratio starts to fall ;

New Data Usage: When a larger block is fetched, it contains new data, but the probability of using this new data decreases.

Removal of Old Data: If a larger block is brought into the cache, some of the existing data in the cache must be removed. This old data may have been accessed frequently, so the probability of needing it again is higher.

Conclusion: If the block size becomes too large, it becomes more difficult to use the new data, and since older data is removed from the cache, the hit ratio decreases. This means that the benefit of the new data may be less compared to the old data that was removed.

Mapping Function

- Determines which cache location the block will occupy.

When one block is read in, another may have to be replaced.

Two constraints affect design:

The more flexible the mapping function, the more complex is the circuitry required to search the cache.

The mapping function in cache design determines where a new block of data will be placed. Two key factors influence this design. **First**, when a new block is loaded, an existing block might need to be replaced. Ideally, this is done to minimize the chance of removing a block that will be needed soon, maximizing the hit ratio. A more flexible mapping function allows for better replacement strategies. However, the **second factor** is that increased flexibility (at mapping func.) also makes the circuitry more complex, as it becomes harder to quickly check if a block is already in the cache.

Replacement Algorithm

Least Recently Used (LRU) Algorithm

- Effective strategy is to replace a block that has been in the cache the longest with no references to it.
- Hardware mechanisms are needed to identify the least recently used block
 - Chooses which block to replace when a new block is to be loaded into the cache.

Write Policy

Decides when the memory write operation happens.

- It Can occur every time the block is updated
- It Can occur when the block is replaced
 - Minimizes write operations
 - Leaves main memory in an obsolete state

I/O TECHNIQUES

Three techniques are possible for I/O operations:

Programmed I/O

Interrupt-Driven I/O

Direct Memory Access (DMA)

When the processor is executing a program and encounters an instruction relating to I/O, it executes that instruction by issuing a command to the appropriate I/O module.

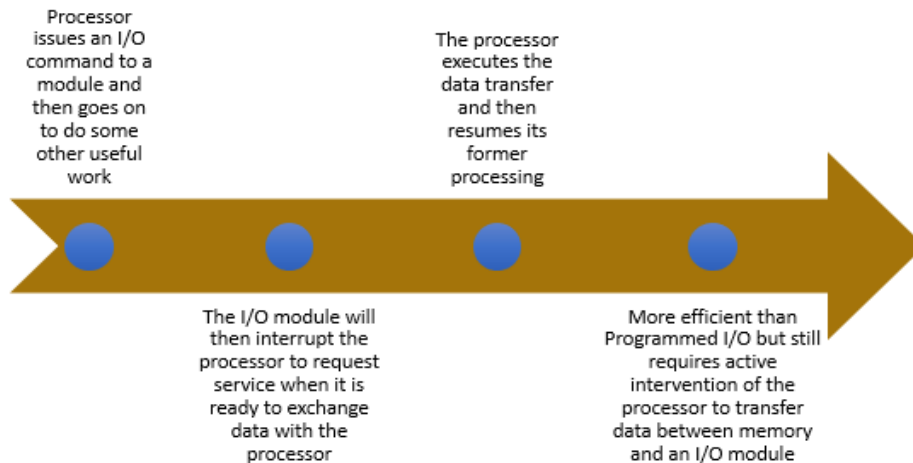
1- Programmed I/O

- The I/O module performs the requested action then sets the appropriate bits in the I/O status register.
- The processor periodically checks the status of the I/O module until it determines the instruction is complete.
- With programmed I/O the performance level of the entire system is severely decreased. (in I/O programmed, the processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of more data)

2- Interrupt-Driven I/O



A different method, called interrupt-driven I/O, is when the processor sends an I/O command to a device and then continues to do other work. The I/O module will then interrupt the processor to request service when it is ready to exchange data with the processor. The processor then executes the data transfer, as before, and then resumes its former processing.



Interrupt-Driven I/O Drawbacks



- Transfer rate is limited by the speed with which the processor can test and service a device
- The processor is tied up in managing an I/O transfer.
- A number of instructions must be executed for each I/O transfer

3- Direct Memory Access(DMA)



When large volumes of data are to be moved, a more efficient technique is required: **direct memory access (DMA)**. The DMA function can be performed by a separate module on the system bus or it can be incorporated into an I/O module.

When the processor wishes to read or write data it issues a command to the DMA module containing:

- Whether a read or write is requested
- The address of the I/O device involved
- The starting location in memory to read/write
- The number of words to be read/written

Basic Working Principle of DMA:

Processor Starts the Operation, Then Continues with Other Tasks:

- When an I/O operation is needed, the processor delegates this task to the DMA module and continues with other tasks.
- The DMA module takes care of transferring data to or from the memory instead of the processor.

DMA Module Works Directly with Memory:

- The DMA module transfers data directly to memory **without involving the processor**. The data is transferred in blocks, one word at a time, either to or from memory.
- When the data transfer is complete, the DMA module **sends an interrupt signal to the processor**. This way, **the processor is only involved at the beginning and end of the**

More efficient than interrupt-driven or programmed I/O

transfer.